

FUNDAMENTALS OF PYTHON



Surbhi Chhabra
Assistant Professor, JKLU, Jaipur

OVERVIEW

- We will explore simple python basics.
 - Beginning with Python, Core Objects
 - Statements, Expressions, Comments
 - Variables
 - Operators
 - Conditional Statements

PRACTICE , PRACTICE , PRACTICE

The best way to improve your programming and problem skills is to practice!

COMPUTATIONAL THINKING

- Part of our lives, though not explicitly
- School arithmetic: addition with carry, long division, finding square roots, lcm ...
- Arrange books alphabetically on the shelf
- Find the quickest route to the bus stop on the main road
- Making it explicit
- How do we describe an algorithm?
 - To another person, to a computer
- Precise notation for conveying instructions
 - Programming language
- C, C++, Java are very heavy
 - A lot of time is spent in digesting syntax
- Python is much simpler to get into
 - Overcome the fear of programming!
- Precise notation for conveying instructions
 - Programming language

PYTHON: WHAT, WHY?

- Python was created by Guido Van Rossum and was first released in 1991.
- It is a high-level, interpreted, interactive and object-oriented scripting language. Python is designed to be highly readable.
- It has a simple syntax that mimics natural language, so it's easier to read and understand. This makes it quicker to build projects, and faster to improve on them.
- It's versatile: Python can be used for many different tasks, from web development to machine learning.
- It's open source, which means it's free to use and distribute, even for commercial purposes.
- Python's archive of modules and libraries—bundles of code that third-party users have created to expand Python's capabilities—is vast and growing.
- Python has a large and active community that contributes to Python's pool of modules and libraries, and acts as a helpful resource for other programmers. The vast support community means that if coders run into a stumbling block, finding a solution is relatively easy; somebody is bound to have encountered the same problem before.

PYTHON FEATURES

no compiling or linking	rapid development cycle
no type declarations	simpler, shorter, more flexible
automatic memory management	garbage collection
high-level data types and operations	fast development
object-oriented programming	code structuring and reuse, C++
embedding and extending in C	mixed language systems
classes, modules, exceptions	"programming-in-the-large" support
dynamic loading of C modules	simplified extensions, smaller binaries
dynamic reloading of C modules	programs can be modified without stopping

PYTHON FEATURES

universal "first-class" object model	fewer restrictions and rules
run-time program construction	handles unforeseen needs, end-user coding
interactive, dynamic nature	incremental development and testing
access to interpreter information	metaprogramming, introspective objects
wide portability	cross-platform programming without ports
compilation to portable byte-code	execution speed, protecting source code
built-in interfaces to external services	system tools, GUIs, persistence, databases, etc.

INTERPRETER AND COMPILER

- Two kinds of programs process high-level languages into low-level languages: **interpreters** and **compilers**. An interpreter reads a high-level program and executes it, meaning that it does what the program says. It processes the program a little at a time, alternately reading lines and performing computations.
- A compiler reads the program and translates it completely before the program starts running. In this context, the high-level program is called the **source code**, and the translated program is called the **object code** or the **executable**. Once a program is compiled, you can execute it repeatedly without further translation.
- Python is considered an interpreted language because Python programs are executed by an interpreter. There are two ways to use the interpreter: **interactive mode** and **script mode**.

INTERACTIVE AND SCRIPT MODE

- In interactive mode, you type Python programs and the interpreter displays the result:
- `>>> 1 + 1`
- `2`
- The chevron, `>>>`, is the **prompt** the interpreter uses to indicate that it is ready. If you type `1 + 1`, the interpreter replies `2`.
- Alternatively, you can store code in a file and use the interpreter to execute the contents of the file, which is called a **script**. By convention, Python scripts have names that end with `".py"`.
- To execute the script, you have to tell the interpreter the name of the file. If you have a script named `"lab1_programs.py"` and you are working in a UNIX command window, you type `python "lab1_programs.py"`. In other development environments, the details of executing scripts are different.
- Working in interactive mode is convenient for testing small pieces of code because you can type and execute them immediately. But for anything more than a few lines, you should save your code as a script so you can modify and execute it in the future.

THE BUILDING BLOCKS OF PROGRAM

- **Input** Get data from the “outside world”. This might be reading data from a file, or even some kind of sensor like a microphone or GPS. In our initial programs, our input will come from the user typing data on the keyboard.
- **Output** Display the results of the program on a screen or store them in a file or perhaps write them to a device like a speaker to play music or speak text.
- **Sequential Execution** Perform statements one after another in the order they are encountered in the script.
- **Conditional Execution** Check for certain conditions and then execute or skip sequence of statements.
- **Repeated Execution** Perform some set of statements repeatedly, usually with some variation.
- **Reuse** Write a set of instructions once and give them a name and then reuse those instructions as needed throughout your program.

VALUES AND TYPES

- A value is one of the basic things a program works with, like a letter or a number. The values we have seen so far are 1, 2, and “Hello, World!” These values belong to different types: 2 is an integer, and “Hello, World!” is a string, so called because it contains a “string” of letters. You (and the interpreter) can identify strings because they are enclosed in quotation marks.

- The print statement:

- `>>> print(4)`

- 4

- If you are not sure what type a value has, the interpreter can tell you.

- `>>> type('Hello, World!')`

`<class 'str'>`

- `>>> type(17)`

`<class 'int'>`

- Strings belong to the type str and integers belong to the type int. Numbers with a decimal point belong to a type called float, because these numbers are represented in a format called floating point.

- `>>> type(3.2)`

`<class 'float'>`

VALUES AND TYPES

- `>>> type('17')`

`<class 'str'>`

- `>>> type('3.2')`

`<class 'str'>`

- When you type a large integer, you might be tempted to use commas between groups of three digits, as in 1,000,000. This is not a legal integer in Python, but it is legal:

- `>>> print(1,000,000)`

- `1 0 0`

Python interprets 1,000,000 as a comma separated sequence of integers, which it prints with spaces between.

VARIABLES

A variable is a name that refers to a value. A variable is a named place in the memory where a programmer can store data and later retrieve the data using the variable “name”.

- An assignment statement creates new variables and gives them values:

- `>>> message = 'My name is Surbhi'`

- `>>> n = 17`

- `>>> pi = 3.1415926535897931`

- To display the value of a variable, you can use a print statement:

- `>>> print(n) : 17`

- `>>> print(pi) : 3.141592653589793`

- `>>> type(message)`

`<class 'str'>`

- `>>> type(n)`

`<class 'int'>`

- `>>> type(pi)`

`<class 'float'>`

PYTHON VARIABLES

- The most basic component of any programming language are "things," also called **variables**.
- A variable is a named place in the memory where a programmer can store data and later retrieve the data using the variable "name".
- Programmers get to choose the names of the variables
- You can change the contents of a variable in a later statement
- Variables can be integers, decimal numbers (floats), words and sentences (string), lists etc. In the below example 'x' and 'y' are variables.

x 17.3

y 59

x 37

new value stored in
'x' is 37 not 17.3

VARIABLES NAMING RULES

- You cannot use one of Python's keywords as a variable name.
- A variable name cannot contain spaces.
- You cannot use a variable until you have assigned a value to it. An error will occur if you try to perform an operation on a variable, such as printing it, before it has been assigned a value.
- The first character must be one of the letters a through z, A through Z, or an underscore character (_). After the first character you may use the letters a through z or A through Z, the digits 0 through 9, or underscores.
- • Uppercase and lowercase characters are distinct. This means the variable name `ItemsOrdered` is not the same as `itemsordered`.
- You should always choose names for your variables that give an indication of what they are used for.
- `units_per_day`: Legal
- `dayOfWeek`: Legal
- `3dGraph`: Illegal. Variable names cannot begin with a digit.
- `June1997`: Legal
- `Mixture#3`: Illegal. Variable names may only use letters, digits, or underscores.

Assignment Statements

- § We assign a value to a variable using the assignment statement (=)
- § An assignment statement consists of an expression on the right-hand side and a variable to store the result

```
x = 6.7 * 58
```

The Augmented Assignment Operators

Operator	Example Usage	Equivalent To
<code>+=</code>	<code>x += 5</code>	<code>x = x + 5</code>
<code>-=</code>	<code>y -= 2</code>	<code>y = y - 2</code>
<code>*=</code>	<code>z *= 10</code>	<code>z = z * 10</code>
<code>/=</code>	<code>a /= b</code>	<code>a = a / b</code>
<code>%=</code>	<code>c %= 3</code>	<code>c = c % 3</code>
<code>//=</code>	<code>x //= 3</code>	<code>x = x // 3</code>
<code>**=</code>	<code>y **= 2</code>	<code>y = y**2</code>

Sentences or Lines

`x = 2`

`x = x + 2`

`print(x)`

Assignment statement

Assignment with expression

Print statement

Variable

Operator

Constant

Function

HOW DO I CREATE A VARIABLE AND ASSIGN IT A VALUE?

- `x = 5`: This creates a variable named `x` with value 5
`5 = x` is not a valid command; variable name needs to be on the left.
- `print(x)`: This prints the value stored in `x` (5 in this case) on the terminal.
- `a = 3`
- `b = 4`
- `c = a + b`
- `print(c)` : Prints 7 on the terminal
- `a = "Hello"`
- `b = " "`
- `c = "World"`
- `print(a+b+c)` : Prints Hello World on the terminal : Concept of Concatenation

LITERALS AND VARIABLES

- Strings like 'My name is Surbhi' and numbers like 17, 2.0 are called literals in Python. Formally, a literal is something that describes a constant value. Variables are containers that are used to store values.

Constants

- Fixed values such as numbers, letters, and strings, are called “constants” because their value does not change

Numeric constants are as you expect

String constants use single quotes (')
or double quotes (")

```
>>> print(123)
123
>>> print(98.6)
98.6
>>> print('Hello world')
```

KEYWORDS

- As of now, python reserves 35 keywords:

and	continue	finally	is	raise
as	def	for	lambda	return
assert	del	from	None	True
async	elif	global	nonlocal	try
await	else	if	not	while
break	except	import	or	with
class	False	in	pass	yield

KEYWORDS

- **and** A logical operator
- **as** To create an alias
- **assert** For debugging
- **break** To break out of a loop
- **class** To define a class
- **continue** To continue to the next iteration of a loop
- **def** To define a function
- **del** To delete an object
- **elif** Used in conditional statements, same as else if
- **else** Used in conditional statements
- **except** Used with exceptions, what to do when an exception occurs
- **False** Boolean value, result of comparison operations
- **finally** Used with exceptions, a block of code that will be executed no matter if there is an exception or not
- **for** To create a for loop
- **from** To import specific parts of a module
- **global** To declare a global variable

KEYWORDS

- **if** To make a conditional statement
- **import** To import a module
- **in** To check if a value is present in a list, tuple, etc.
- **is** To test if two variables are equal
- **lambda** To create an anonymous function
- **None** Represents a null value
- **nonlocal** To declare a non-local variable
- **not** A logical operator
- **or** A logical operator
- **pass** A null statement, a statement that will do nothing
- **raise** To raise an exception
- **return** To exit a function and return a value
- **True** Boolean value, result of comparison operations
- **try** To make a try...except statement
- **while** To create a while loop
- **with** Used to simplify exception handling
- **yield** To end a function, returns a generator

User Input

We can instruct Python to pause and read data from the user using the `input()` function

The `input()` function returns a string

```
a = input('Who are you? ')  
print('Welcome', a)
```

Who are you? Surbhi
Welcome Surbhi

Numeric Expressions

+ Addition Adds two numbers

- Subtraction Subtracts one number from another

* Multiplication Multiplies one number by another (Asterisk is multiplication)

/ Division Divides one number by another and gives the result as a floating-point number

// Integer division Divides one number by another and gives the result as a whole number

% Remainder Divides one number by another and gives the remainder

** Exponent Raises a number to a power

Operator	Operation
+	Addition
-	Subtraction
*	Multiplication
/	Division
**	Power
//	Floor Division
%	Remainder

Order of Evaluation

When we string operators together - Python must know which one to do first

This is called “operator precedence”

Which operator “takes precedence” over the others?

$$x = 1 + 2 * 3 - 4 / 5 ** 6$$

Operator Precedence Rules

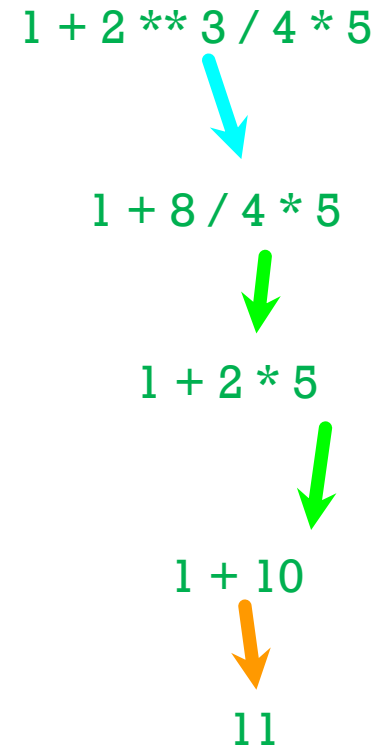
Highest precedence rule to lowest precedence rule:

- Parenthesis are always respected
- Exponentiation (raise to a Power)
- Multiplication, Division, and Remainder
- Addition and Subtraction
- Left to right

Parenthesis
Power
Multiplication/Division
Addition/Subtraction
Left to Right

```
>>> x = 1 + 2 ** 3 / 4 * 5
>>> print(x)
11.0
```

Parenthesis
Power
Multiplication
Addition
Left to Right



Operator Precedence : Rules

- Remember the rules top to bottom
- When writing code - use parentheses
- When writing code - keep mathematical expressions simple enough that they are easy to understand
- Break long series of mathematical operations up to make them more clear
- Notice the multiplication (*), floating-point division (/), integer division (/), and remainder (%) operators have the same precedence. The addition (+) and subtraction (−) operators also have the same precedence. When two operators with the same precedence share an operand, the operators execute from left to right.
- There is an exception to the left-to-right rule. When two ** operators share an operand, the operators execute right-to-left. For example, the expression `2**3**4` is evaluated as `2**(3**4)`.

Grouping with Parentheses

- Parts of a mathematical expression may be grouped with parentheses to force some operations to be performed before others.
- In the following statement, the variables `a` and `b` are added together, and their sum is divided by 4:
`result = (a + b) / 4`
- Without the parentheses, however, `b` would be divided by 4 and the result added to `a`.

Expression	Value
• $(5 + 2) * 4$	28
• $10 / (5 - 3)$	5.0
• $8 + 12 * (6 - 2)$	56
• $(6 - 3) * (2 + 7) / 3$	9.0

PYTHON DATA TYPES

Python has the following data types built-in by default, in these categories:

- **Numeric Types:**

- ☐ **Int :** `x = -5, 0, 1000000`
- ☐ **Float :** `x = -2.0, 3.14159, 453.234`
- ☐ **Complex:** `x=2+7j`

- **Boolean Type:**

- ☐ **Boolean :** `x = True, False`

- **Text/String Type:**

- ☐ **Str :** `x = "Hello world!", "K3WL"`

- **Sequence Type:**

- ☐ **List :** `x = [1, 2, 3, 4], ["Hello", "world!"], [1, "Hello", True, 0.2]`
- ☐ **Tuple:** `x = ("apple", "banana", "cherry")`
- ☐ **Range:** `x = range(6)`

- **Mapping Type:**

- ☐ **Dict:** `x = {"name" : "John", "age" : 36}`

- **Set Type:**

- ☐ **Set** `x = {"apple", "banana", "cherry"}`
- ☐ **Frozen Set:** `x = frozenset({"apple", "banana", "cherry"})`

- **Binary Type:**

- ☐ **bytes:** `x = b"Hello"`
- ☐ **bytearray:** `x = bytearray(5)`
- ☐ **memoryview:** `x = memoryview(bytes(5))`

- **None Type:**

- ☐ **NoneType:** `x = None`

Comments in Python

Anything after a # is ignored by Python

Why comment?

- Describe what is going to happen in a sequence of code
- Document who wrote the code or other important information
- Turn off a line of code - perhaps temporarily
- (Three Single Quotes) : For multiline comments use ``` From where you want to start the comment and also end with ```

What Does “Type” Mean?

In Python variables, literals, and constants have a “type”

Python knows the difference between an integer number and a string

For example “+” means “addition” if something is a number and “concatenate” if something is a string

```
>>> x = 1 + 4
>>> print(x)
5
>>> message = 'hello ' + 'surbhi'
>>> print(message)
hello surbhi
```

concatenate = put together

Type Matters

Python knows what “type” everything is

- You cannot “add 1” to a string

We can ask Python what type something is by using the `type()` function

```
>>> message = 'hello ' + 'surbhi'
>>> message = message + 1
Traceback (most recent call last): File
"<stdin>", line 1, in <module>TypeError:
Can't convert 'int' object to str implicitly
>>> type(message)
<class'str'>
>>> type('hello')
<class'str'>
>>> type(1)
<class'int'>
>>>
```

Several Types of Numbers

Numbers have two main types

- Integers are whole numbers:
-14, -2, 0, 1, 100, 401233
- Floating Point Numbers have decimal parts: -2.5 , 0.0, 98.6, 14.0

```
>>> x = 1
>>> type(x)
<class 'int'>
>>> temp = 98.6
>>> type(temp)
<class'float'>
>>> type(1)
<class 'int'>
>>> type(1.0)
<class'float'>
>>>
```

Type Conversions

When you put an integer and floating point in an expression, the integer is implicitly converted to a float

You can control this with the built-in functions `int()` and `float()`

```
>>> print(float(99) + 100)
199.0
>>> i = 42
>>> type(i)
<class'int'>
>>> f = float(i)
>>> print(f)
42.0
>>> type(f)
<class'float'>
```

Logical Operators

- **And:** Both subexpressions must be true for the compound expression to be true.
- **Or:** One or both subexpressions must be true for the compound expression to be true. It is only necessary for one of the subexpressions to be true, and it does not matter which.
- **Not:** The not operator is a unary operator, meaning it works with only one operand. The operand must be a Boolean expression. The not operator reverses the truth of its operand. If it is applied to an expression that is true, the operator returns false. If it is applied to an expression that is false, the operator returns true.

Expression	Meaning
<code>x > y and a < b</code>	Is x greater than y AND is a less than b?
<code>x == y or x == z</code>	Is x equal to y OR is x equal to z?
<code>not (x > y)</code>	Is the expression <code>x > y</code> NOT true?

```
if temperature < 20 and minutes > 12:  
    print('The temperature is in the danger zone.')
```

Expression

true and false
false and true
false and false
true and true

Value of the Expression

false
false
false
true

```
if temperature < 20 or minutes > 12:  
    print('The temperature is too extreme.')
```

Expression

true or false
false or true
false or false
true or true

Value of the Expression

true
true
false
true

```
if not(temperature > 100):  
    print('This is below the maximum temperature.')
```

Expression

Not true
Not false

Value of the Expression

false
true

Boolean Variables

- A Boolean variable can reference one of two values: **True** or **False**.
- Boolean variables are commonly used as flags, which indicate whether specific conditions exist.
- A flag is a variable that signals when some condition exists in the program. When the flag variable is set to **False**, it indicates the condition does not exist. When the flag variable is set to **True**, it means the condition does exist.

```
Sales=float(input("Enter Sales:"))
```

```
if sales >= 50000.0:
```

```
    sales_quota_met = True
```

```
else:
```

```
    sales_quota_met = False
```

```
if sales_quota_met:
```

```
    print('You have met your sales quota!')
```

```
if sales_quota_met == True:
```

```
    print('You have met your sales quota!')
```

} Same Expression

```

print("Hello Students")
print("Welcome\n How are you all doing?") #\n means new line
print("Welcome\tHow are you all doing")#\t inserts a tab space.
print(2)
print(2,4,6,8)#By default, print() separates multiple items with a space
print(2,4,6,8, sep="")#No separator
print(2,4,6,8, sep="-")#Custom separator (-).
print(print)#you're printing the print function object itself, not executing it
print("\"Hello\"")#To print quotes inside text, you escape them with \"
print("Hello \"Students\"")
print("Hello \\Students")#\ is used to print a single backslash
print('"Hello"')
print('"Hello \\Students"')
#print=5
#print+print

type(2)# int (integer)
type(2.4)
type("Hello Students")
type(True)
type(False)
type(2.)# float (same as 2.0)
type(None)# NoneType (represents "no value")
type(2.3%3)# float (because % gives remainder; 2.3 % 3 = 2.3 since 2.3 < 3)

```

```
In [6]: print("Hello Students")
Hello Students

In [7]: print("Welcome\n How are you all doing?")
Welcome
How are you all doing?

In [8]: print("Welcome\tHow are you all doing")
Welcome How are you all doing

In [9]: print(2)
2

In [10]: print(2,4,6,8)
2 4 6 8

In [11]: print(2,4,6,8, sep="")
2468

In [12]: print(2,4,6,8, sep="-")
2-4-6-8

In [13]: print(print)
<built-in function print>

In [14]: print("\nHello\n")
"Hello"

In [15]: print("Hello \"Students\"")
Hello "Students"

In [16]: print("Hello \\Students")
Hello \Students
```

```
In [17]: print('"Hello"')
Hello

In [18]: print('"Hello \Students"')
Hello \Students

In [19]: type(2)
Out[19]: int

In [20]: type(2.4)
Out[20]: float

In [21]: type("Hello Students")
Out[21]: str

In [22]: type(True)
Out[22]: bool

In [23]: type(False)
Out[23]: bool

In [24]: type(2.)
Out[24]: float

In [25]: type(None)
Out[25]: NoneType

In [26]: type(2.3%3)
Out[26]: float
```

```
In [27]: 10 + 5
Out[27]: 15

In [28]: 10-5
Out[28]: 5

In [29]: 10/5
Out[29]: 2.0

In [30]: 10*5
Out[30]: 50

In [31]: 10//5
Out[31]: 2

In [32]: 10%5
Out[32]: 0

In [33]: 10**5
Out[33]: 100000

In [34]: +10
Out[34]: 10

In [35]: -10
Out[35]: -10

In [36]: 10 --5
Out[36]: 15

In [37]: -10+5
Out[37]: -5
```

```
In [39]: 10>5
Out[39]: True

In [40]: 10>=5
Out[40]: True

In [41]: 10<5
Out[41]: False

In [42]: 10<=5
Out[42]: False

In [43]: 10==5
Out[43]: False

In [44]: 10!=5
Out[44]: True

In [45]: x=10, y=5
         Cell In[45], line 1
           x=10, y=5
           ^
SyntaxError: invalid syntax.

In [46]: x,y=10,5

In [47]: z=y>x

In [48]: print(z)
False
```

```
In [49]: 3 < 3.14 and 4 > 3.14
Out[49]: True

In [50]: 3 < 3.14 < 4
Out[50]: True

In [51]: 10 > 20 or True
Out[51]: True

In [52]: False or False or False or False or True
Out[52]: True

In [53]: False or False or False or False or True
Out[53]: True

In [54]: not True and False
Out[54]: False

In [55]: not(True) and False
Out[55]: False

In [56]: not(True and False)
Out[56]: True

In [57]: True or False and False
Out[57]: True

In [58]: (True or False) and False
Out[58]: False

In [59]: True or (False and False)
Out[59]: True
```

```
In [60]: 10.000000000000000000000000000001 > 10  
Out[60]: False
```

```
In [61]: 0.1 ** 100 == 0.0  
Out[61]: False
```

```
In [62]: 0.1 ** 1000 == 0.0  
Out[62]: True
```

```
In [63]: 0.1 * 3 == 0.3  
Out[63]: False
```

```
In [64]: 0.1 * 3  
Out[64]: 0.30000000000000004
```

```
In [65]: 10.000000000000000000000000000001 >= 10  
Out[65]: True
```

Floating Point Precision

How Python stores 0.1

0.1 **cannot be represented exactly** in binary floating point.

Instead, Python stores something like: **0.10000000000000000555...**

2. Multiplying by 3

When you compute $0.1 * 3$, the internal operation is really:

$0.10000000000000000555... \times 3 = 0.30000000000000001665...$

3. Comparing to 0.3

0.3 is also **not exact** in binary, stored as: **0.29999999999999998889...**

So: $0.1 * 3 = 0.300000000000000004$

$0.3 = 0.29999999999999999$

They differ by a tiny amount, hence:

$0.1 * 3 == 0.3$ # False

Short Circuit Evaluation

```
In [1]: 1 / 0
```

```
ZeroDivisionError
```

Traceback (most recent call last)

```
Cell In[1], line 1
```

```
----> 1 1 / 0
```

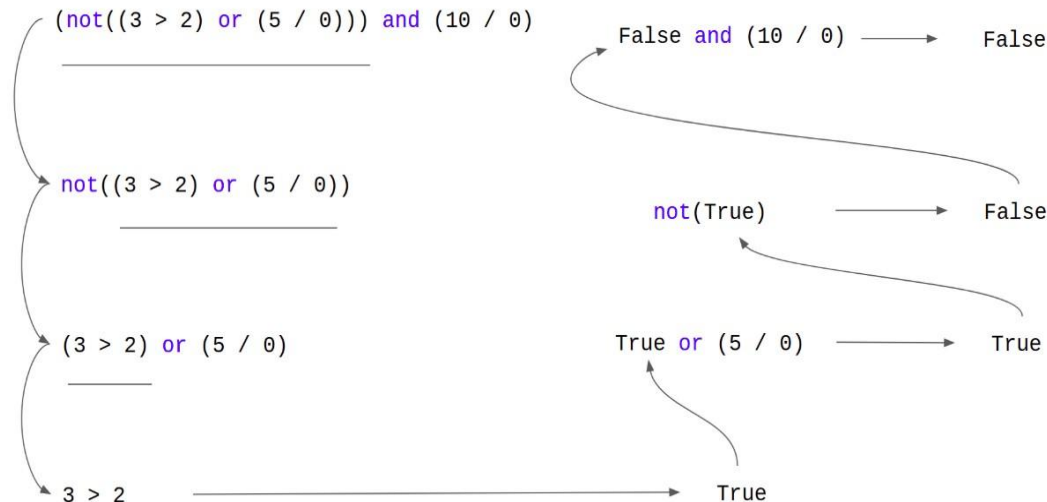
```
ZeroDivisionError: division by zero
```

```
In [2]: True or (1 / 0)
```

```
Out[2]: True
```

```
In [3]: (not((3 > 2) or (5 / 0))) and (10 / 0)
```

```
Out[3]: False
```



Python Bit wise Operators

- They instruct the interpreter of Python to perform operations at a bit level. This is known as binary manipulation.
- Every integer in Python is stored in Binary format. These bitwise operators are the only ones among the Python operators that let you manipulate individual bits directly.
- Note: Unlike lower-level languages like C or Java, Python supports arbitrary-length integers, so you're not limited to 8, 16, or 32 bits. You can perform Python bitwise operations on very large numbers without worrying about overflow. This makes bit manipulation in Python highly flexible.

Python Bit wise Operators

Operator	Name	Example	Description
&	Bitwise AND	a & b	Sets each bit to 1 only if both bits are 1
	Bitwise OR	a b	Sets each bit to 1 if at least one bit is 1
^	Bitwise XOR	a ^ b	Sets each bit to 1 only if the bits are different
~	Bitwise NOT	~a	Inverts all bits (1s become 0s and 0s become 1s)
<<	Left Shift	a << 1	Shifts bits to the left, filling with 0s on the right
>>	Right Shift	a >> 1	Shifts bits to the right, discarding bits on the right

Bitwise AND (&) Operator in Python

Bit A	Bit B	A & B
0	0	0
0	1	0
1	0	0
1	1	1

Bitwise OR Operator in Python

Bit A	Bit B	A B
0	0	0
0	1	1
1	0	1
1	1	1

Bitwise XOR (^) Operator in Python

Bit A	Bit B	A ^ B
0	0	0
0	1	1
1	0	1
1	1	0

Bitwise NOT Operator in Python

Bit A	$\sim A$
0	1
1	0

Left Shift (<<) Bitwise Operator in Python

- This operator shifts the bits in the binary representation of 'a' to the left by 'n' positions. A zero is added to the right, and the leftmost bits are discarded.
- This operation is equivalent to multiplying the number by 2^n .

Right Shift (>>) Bitwise Operator in Python

- This operator shifts the bits towards the right by n positions. Here, the rightmost bits are discarded, and the leftmost bits are filled based on the sign of the number.
- For a Positive Number, 0 is filled on the left.
- For a Negative Number, 1's are filled on the left to preserve the sign. This is called an arithmetic shift.
- It performs floor division by 2^n .

```

In [72]: a = 10
...: b = 4
...: print(a & b)
0

In [73]: a = 10
...: b = 4
...: print(a | b)
14

In [74]: a = 10
...: b = 4
...: print(a ^ b)
14

In [75]: a = 10
...: print(~a)
-11

In [76]: a = 5
...: print(a << 1)
10
...: print(a << 2)
20

In [77]: a = 5
...: b = -5
...: print(a >> 1)
2
...: print(b >> 1)
-3

In [78]: a = -5
...: print(a << 1)
-10
...: print(a << 2)
-20

```

Difference Between Bitwise and Logical Operators in Python

Aspect	Python Bitwise Operators	Python Logical Operators
Used For	Performing operations on bits of integers	Evaluating Boolean expressions
Operands	Works on integers	Works on boolean values (True or False)
Operators	& (AND), (OR), ^ (XOR), ~ (NOT), <<, >>	&&, , !
Operation Level	Performs operation bit by bit	Performs operation on the overall truth value
Return Type	Returns an integer result	Returns a boolean (True or False)
Example	5 & 3 → 1 (101 & 011 = 001)	True and False → False
Common Use Cases	Bit masking, low-level hardware operations, and flags	Conditional logic, control flow

```
In [18]: print(3&3)
...: print(3<<2)
...: print(3&3<<2)
...: print((3&3)<<2)
...: print((3&3)>>1)
...:
...: x,y=5,10
...: print(x and y)
...:
...: print(bool(0))
...: print(bool(1))
...: print(bool(""))
...: print(bool("1"))
...: print(bool([1]))

3
12
0
12
1
10
False
True
False
True
True
```

Identity Operators

Identity operators are used to compare the objects, not if they are equal, but if they are actually the same object, with the same memory location:

Operator	Description	Example
is	Returns True if both variables are the same object	x is y
is not	Returns True if both variables are not the same object	x is not y

```
x = [1, 2, 3]
y = [1, 2, 3]
z = x
print(x is y)    # False → different objects, even though values are same
print(x == y)    # True  → values are equal
print(x is z)    # True  → same object in memory
print(x is not y) # True  → different objects
```

Membership Operators

Membership operators are used to test if a sequence is present in an object

Operator	Description	Example
in	Returns True if a sequence with the specified value is present in the object	x in y
not in	Returns True if a sequence with the specified value is not present in the object	x not in y

```
nums = [10, 20, 30, 40]
```

```
text = "hello"
```

```
print(20 in nums)    # True
```

```
print(25 in nums)    # False
```

```
print("h" in text)   # True
```

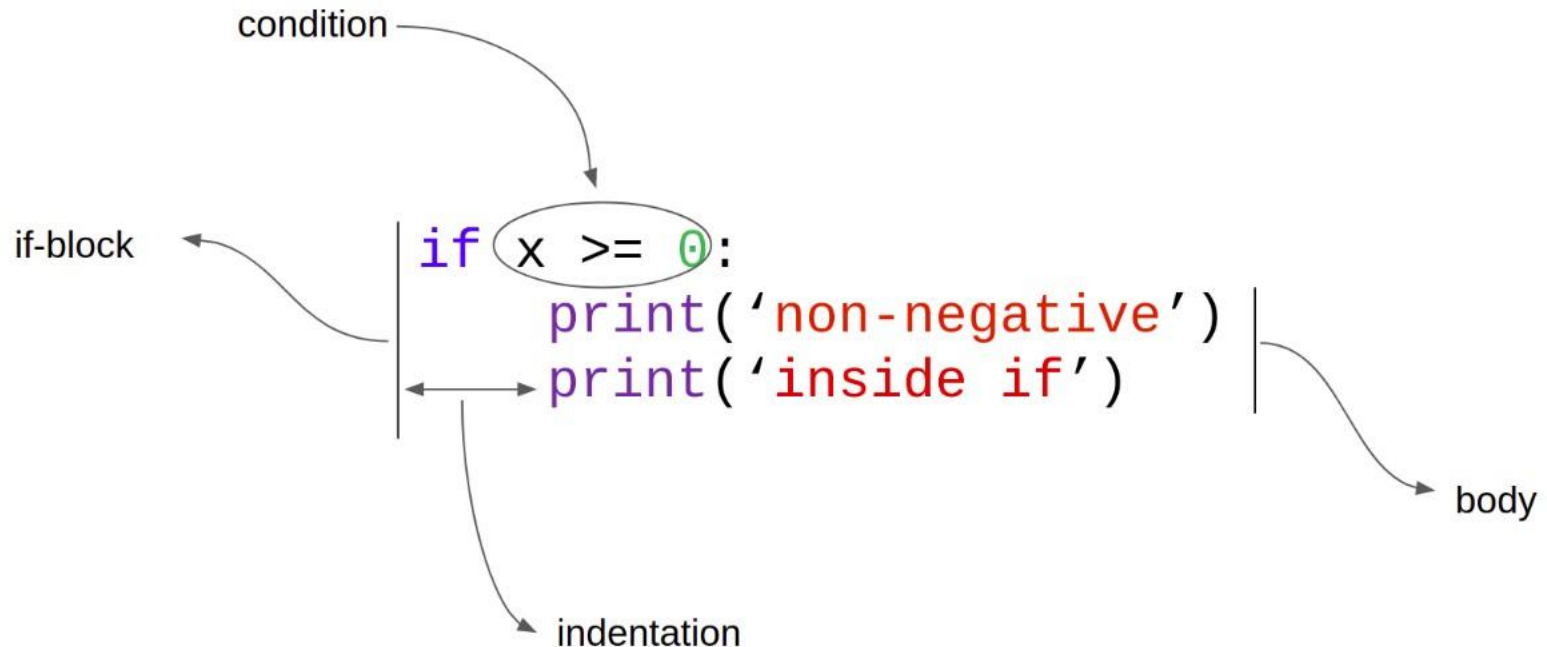
```
print("z" not in text) # True
```

Operator Precedence

Operators	Meaning
()	Parentheses
**	Exponent
+X, -X, ~X	Unary plus, Unary minus, Bitwise NOT
*, /, //, %	Multiplication, Division, Floor division, Modulus
+, -	Addition, Subtraction
<<, >>	Bitwise shift operators
&	Bitwise AND
^	Bitwise XOR
	Bitwise OR
==, !=, >, >=, <, <=, is, is not, in, not in	Comparisons, Identity, Membership operators
not	Logical NOT
and	Logical AND
or	Logical OR

Conditional Statements: if

Problem: Accept an integer as input from the user. If the number is greater than or equal to zero, print: non-negative.



Conditional Statements: if-else

Problem: Accept an integer as input from the user. If the number is greater than or equal to zero, print: non-negative. If the number is less than zero, print negative..

```
x = int(input())
if x >= 0:
    print('non-negative')
else:
    print('negative')
```

`else` is a keyword in Python. When the if-condition evaluates to `True`, the statements inside the body of the if-block are evaluated. When the condition evaluates to `False`, the statements inside the body of the else-block are evaluated

Conditional Statements: if-elif-else

Problem: Accept an integer as input from the user. If the number is greater than zero, print: positive. If the number is less than zero, print negative. If the number is equal to zero, print zero.

```
x = int(input())
if x > 0:
    print('positive')
elif x == 0:
    print('zero')
else:
    print('negative')
# End of code
```

elif is a keyword in Python. It is a shorthand for else-if.

Conditional Statements: if-elif-else

```
if <condition-1>:  
    <statement-1>  
elif <condition-2>:  
    <statement-2>  
else:  
    <statement-3>
```

Some features to note:

- Exactly one of the three statements gets executed.
- The moment either an `if` or an `elif` condition evaluates to `True`, the body of that block is executed and the flow exits out of the entire `if-elif-else` block.
- There could be multiple `elif` conditions after the `if`.
- An `else` condition cannot come before an `elif`. The final `else` block is not mandatory and can be removed. If the `else` is present, it can only come at the end.

Nested Conditional Statements

Problem: Accept three distinct integers as input from the user. If the numbers have been entered in ascending order, print in ascending order. If not, print not in ascending order.

```
# Incomplete solution
```

```
x = int(input())
```

```
y = int(input())
```

```
z = int(input())
```

```
if x < y:
```

```
    print('in ascending order')
```

```
else:
```

```
    print('not in ascending order')
```

The problem with the above solution is that it doesn't check if $y < z$. So, for an input like $x, y, z = 1, 3, 2$, it will print in ascending order, which is incorrect.

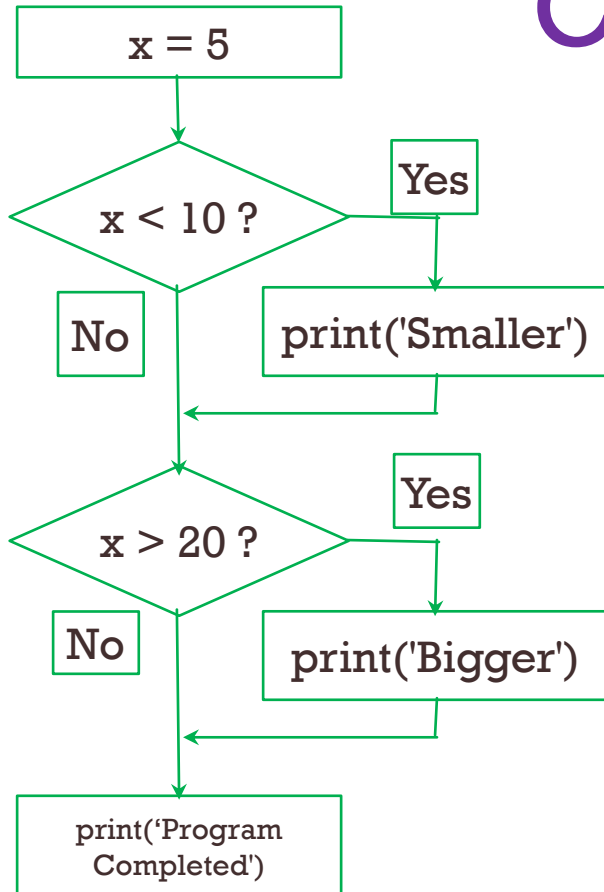
Nested Conditional Statements

The complete solution is given below:

```
x = int(input())
y = int(input())
z = int(input())

if x < y:
    if y < z:
        print('in ascending order')
    else:
        print('not in ascending order')
else:
    print('not in ascending order')
```

Conditional Statements



```
x = 5
if x < 10:
    print('Smaller')
if x > 20:
    print('Bigger')

print('Program Completed')
```

Smaller
Program
Completed

Comparison Operators

- Boolean expressions ask a question and produce a **Yes or No** result which we use to control program flow
- Boolean expressions using comparison operators evaluate to **True / False** or **Yes / No**

Comparison operators look at variables but do not change the variables

Python	Meaning
<	Less than
<=	Less than or Equal to
==	Equal to
>=	Greater than or Equal to
>	Greater than
!=	Not equal

Remember: “=” is used for assignment.

Comparison Operators

```
if x == 5 :  
    print('Equals 5')  
if x > 4 :  
    print('Greater than 4')  
if x >= 5 :  
    print('Greater than or Equals 5')  
if x < 6 : print('Less than 6')  
if x <= 5 :  
    print('Less than or Equals 5')  
if x != 6 :  
    print('Not equal 6')
```

Equals 5

Greater than 4

Greater than or Equals 5

Less than 6

Less than or Equals 5

Not equal 6

Indentation

- Increase indent after an if statement or for statement (after :)
- Maintain indent to indicate the scope of the block (which lines are affected by the if/for)
- Reduce indent back to the level of the if statement or for statement to indicate the end of the block
- Comments on a line by themselves are ignored with regard to indentation

Nested Decisions

```
if x > 1 :  
    print('More than one')  
    if x < 100 :  
        print('Less than 100')  
print('All done')
```

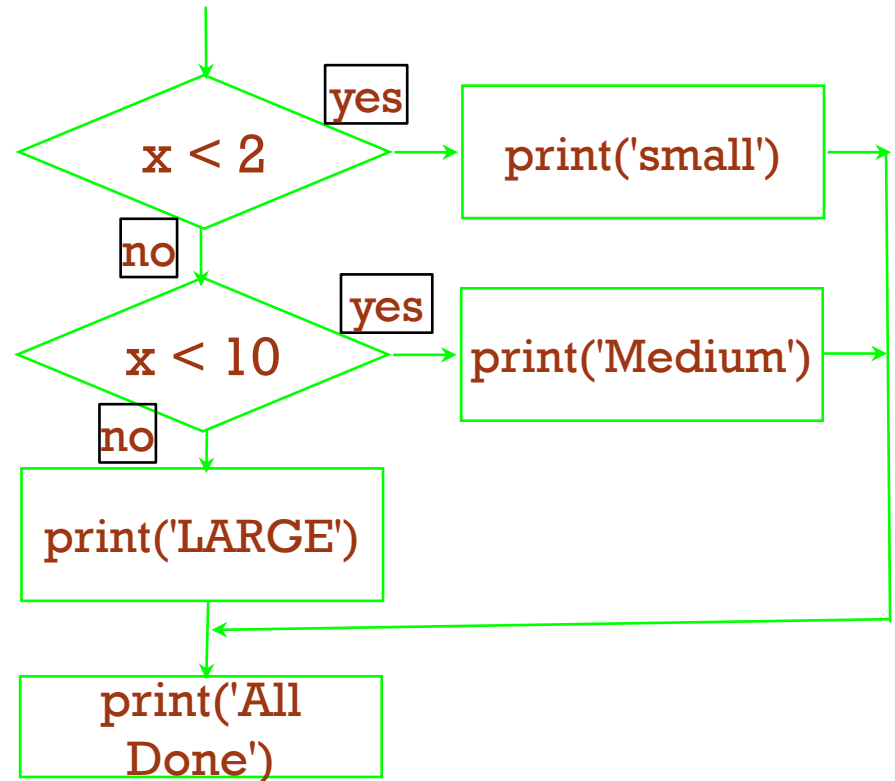
Two-Way Decisions with else:

```
x = 4
if x > 2 :
    print('Bigger')
else :
    print('Smaller')

print('All done')
```

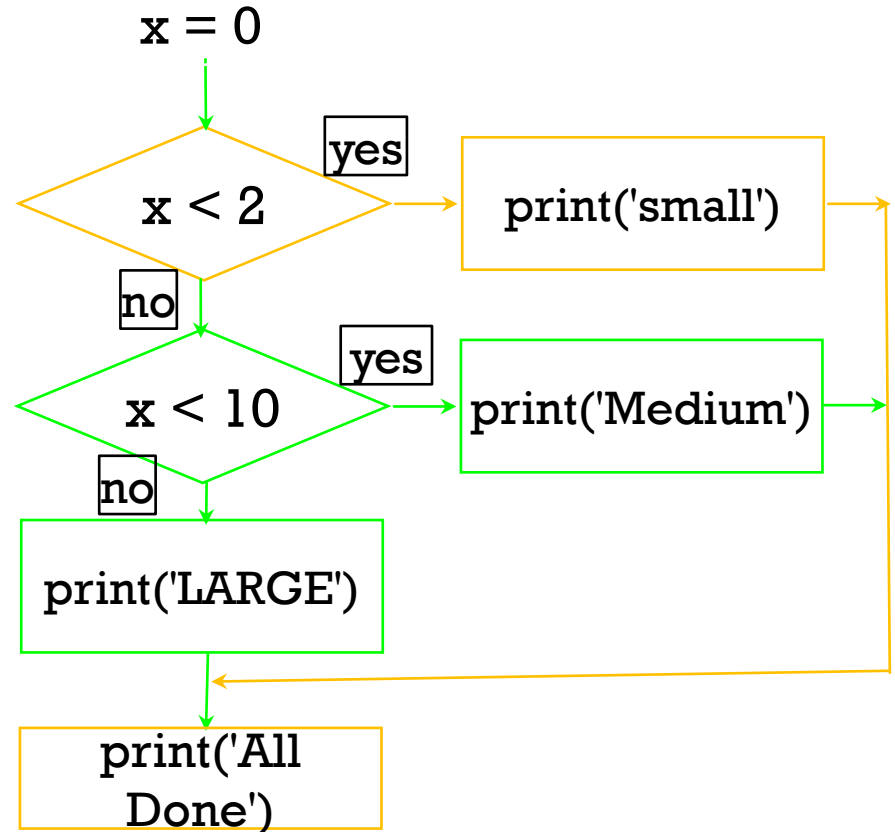
Multi-way

```
if x < 2 :  
    print('small')  
elif x < 10 :  
    print('Medium')  
else :  
    print('LARGE')  
print('All done')
```



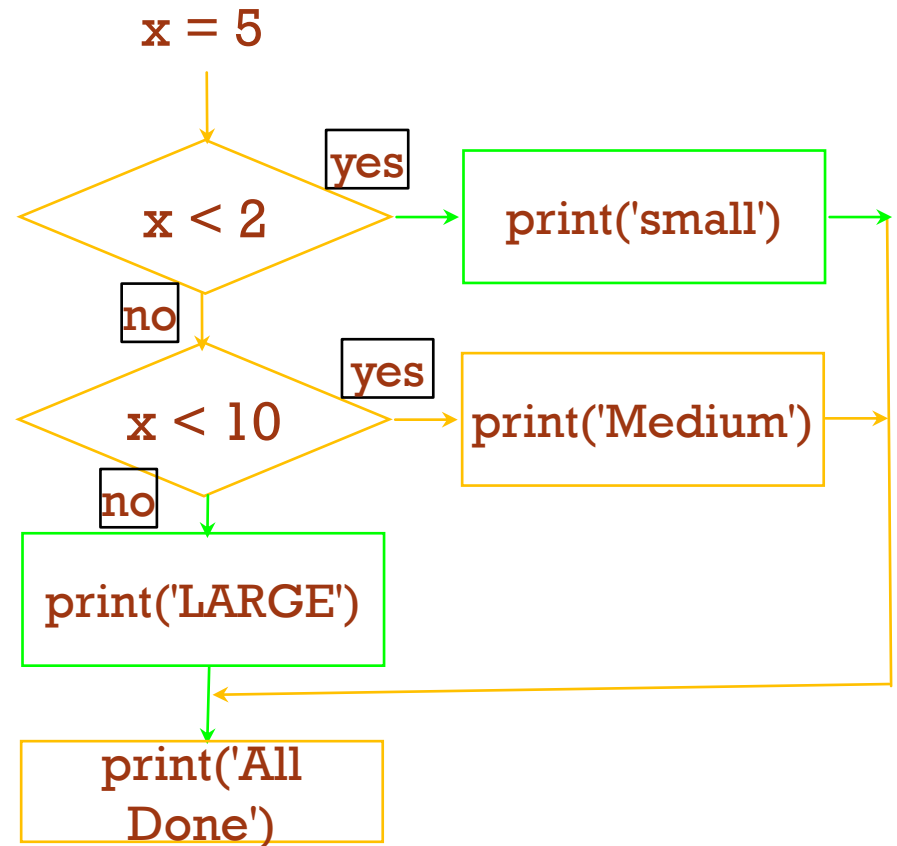
Multi-way

```
x = 0
if x < 2 :
    print('small')
elif x < 10 :
    print('Medium')
else :
    print('LARGE')
print('All done')
```



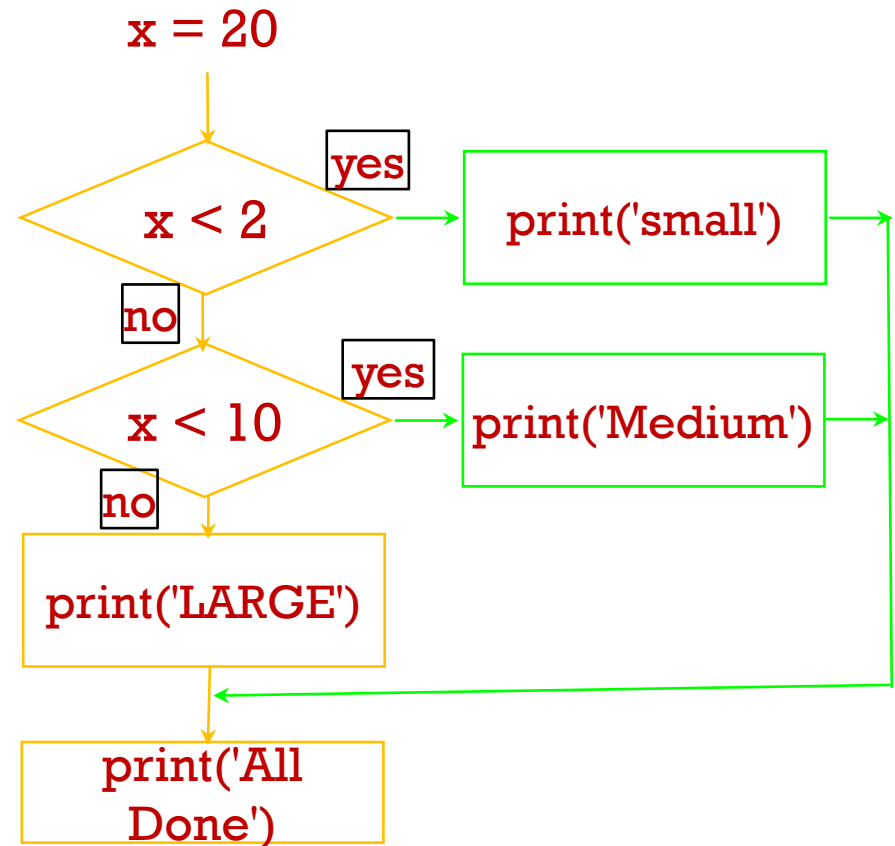
Multi-way

```
x = 5
if x < 2 :
    print('small')
elif x < 10 :
    print('Medium')
else :
    print('LARGE')
print('All done')
```



Multi-way

```
x = 20
if x < 2 :
    print('small')
elif x < 10 :
    print('Medium')
else :
    print('LARGE')
print('All done')
```



Multi-way

No Else

```
x = 5
if x < 2 :
    print('Small')
elif x < 10 :
    print('Medium')

print('All done')
```

#With Else

```
if x < 2 :
    print('Small')
elif x < 10 :
    print('Medium')
elif x < 20 :
    print('Big')
elif x < 40 :
    print('Large')
elif x < 100:
    print('Huge')
else :
    print('Extremely Large')
```